

Measuring Validator and Sandboxed-Execution Coverage for LLM-Generated NetOps Artifacts: A Paired Confirmatory Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e) that measured the marginal detection benefit of adding sandboxed emulation to a deterministic validator for LLM-generated intent→configuration artifacts. Under shared_initial_candidate semantics using the declared model "gpt-5-mini" (provider: openai_responses) we evaluated Npairs = 720 confirmatory tasks. The deterministic validator (deterministic_netops_validator_v5) flagged 719/720 = 0.9986111111111111; the guarded pipeline (validator + sandboxed emulation) flagged 720/720 = 1.0. Paired difference (guarded - baseline) = 0.001388888888888884; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI (10000 resamples, seed = 1729) = [0.0, 0.004166666666666667]. Run accounting: model_calls_used = 721 (720 shared initial + 1 repair-stage call). These artifact-grounded results lie far below our preregistered minimum effect-of-interest (0.20); we report the preregistered design, exact quantitative outcomes, run-accounting diagnostics, artifact-grounded interpretation for the negligible guarded benefit in this regime, and reproducibility pointers into the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration and NetOps workflows. Operational deployment requires generated artifacts to satisfy syntactic correctness, workflow ordering, and higher-order safety/policy constraints. A common mitigation architecture is generate→validate→(repair)→execute, sometimes augmented by sandboxed emulation to reveal runtime-only failures that static validators miss. Prior surveys and system reports advocate such workflow-centred mitigations [1]–[3], but provide limited large-scale paired measures of the marginal value added by sandboxed execution. Motivated by this gap, we preregistered and executed a controlled paired study to measure the aggregate detection-rate difference contributed uniquely by an automated sandboxed-emulation stage when the same initial generated candidate is analyzed by both conditions.

Research question and contributions. Our preregistered research question was: for synthetic intent→configuration tasks under shared_initial_candidate semantics, what is the paired detection-rate difference (guarded - baseline) between (A) a deterministic validator-only pipeline and (B) the same deterministic validator followed by automated sandboxed emulation? Contributions: (1) a machine-readable preregistration and faithful autonomous execution; (2) an artifact-grounded

paired analysis on Npairs = 720 with complete accounting; (3) an artifact-grounded interpretation explaining a near-zero marginal effect; (4) concise reproducibility pointers into the archived bundle and prescriptive boundary conditions where guarded stages are likely to matter.

II. RELATED WORK

Workflow-centred mitigations and verification tools. Recent surveys and architecting reports document a common pattern for LLM-enabled NetOps: pair generation with deterministic validators, formal verifiers, or emulation to reduce unsafe or incorrect device-level actions [1], [2]. Tool- and system-focused work demonstrates concrete instantiations: configuration-analysis engines (e.g., Batfish-style approaches and NetKAT-based verifiers) provide rule-driven semantic checks over device configurations and have been extended in prototypes that combine generation with symbolic or executable verification [2], [3]. Parallel lines of work emphasize the role of sandboxed execution (Mininet/Mininet+GNS3 hybrids, lightweight emulators) to surface dynamic, stateful failures that static validators cannot express, particularly for make-before-break or multi-step state transitions [4], [5].

Coverage heterogeneity and verifier ceilings. Prior empirical and methodological studies highlight that validator gains depend strongly on the validator rule-set, the task abstraction, and the emulator fidelity. For instance, verification-oriented projects show that many common configuration errors (syntactic mistakes, missing required commands, simple ordering violations) are well-covered by high-quality static validators, whereas subtle runtime invariants or environment-dependent failures (timing-dependent race conditions, stateful interaction errors) often escape static analyses and require emulation or more expressive specification languages [2], [6]. This heterogeneity creates two operationally important regimes: (i) validator-dominated regimes where a broad, rule-rich validator attains a high true-positive rate and leaves little headroom for emulation to add detections, and (ii) runtime-detection regimes where emulators or high-fidelity execution are necessary to reveal violations that are intrinsically dynamic.

LLM interaction patterns, mistake localization, and repair. Empirical studies in adjacent program- and specification-generation tasks show a recurring pattern: LLMs may fail to locate their own reasoning errors reliably but can perform targeted corrections when errors or locations are externally

signalled or when a verifier pinpoints a violation [5], [7]. This motivates generate→validate→repair pipelines that use validators or emulators as external mistake detectors and—optionally—invoke constrained repair-stage model calls. The present study enforces `shared_initial_candidate` semantics to isolate exactly these downstream contributions (validator vs. emulator vs. explicitly recorded repair calls) and avoid conflating generation variability with guarded-stage effects.

Benchmarks, evaluation gaps, and operational implications. Several surveys and position papers argue that existing LLM benchmarks insufficiently capture workflow-centred safety properties (rollback-aware scoring, multi-device transactional integrity, emulation-confirmed invariants) and call for niche benchmarks that combine intent→generate, static verification, and sandboxed execution to reflect operational trustworthiness [1], [6]. Our confirmatory paired design directly addresses this suggested evaluation axis by quantifying the marginal detection contribution of an emulation stage under tightly controlled shared-candidate semantics; the negligible aggregate gain we observed is consistent with a validator-ceiling interpretation in a curated synthetic task bank and illustrates the evaluation-risk of reporting single-stage improvements without workflow-centred controls.

III. METHODOLOGY

Preregistration and experimental design. We preregistered a confirmatory paired-binary design (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e). Primary estimand: `paired_success_rate_difference_guarded_minus_baseline` aggregated across confirmatory samples. Planned confirmatory sample: `Npairs = 720` (planned episodes = 1440). Minimum effect-of-interest for power planning = 0.20 (20 percentage points). The preregistration specified inclusion/exclusion rules, missingness handling, multiplicity handling for exploratory secondary tests, and deterministic contamination controls.

Task bank, vendors, and inclusion. Confirmatory items were synthetic intent→configuration tasks from `intent_configuration_repair_v1` covering three abstracted vendor-CLI families and three topology templates (leaf-spine, 3-node service chain, triangle). Inclusion required vendor-specific parser acceptance; the preregistered missingness policy permitted up to two deterministic reseeds for parser failures; irrecoverable parser failures would be deterministically excluded and replaced. In execution there were zero irrecoverable parser exclusions: `complete_pair_count = 720` (see `analysis/missingness_summary.csv`).

Model configuration and sampling semantics. Declared/requested model: "gpt-5-mini" (provider field: `openai_responses`). The archived model configuration records `max_output_tokens = 2000` and `temperature = null` (`execution/model_configuration.json`, sha256 a40e96e212ab87da...). Important clarification (preregistered and enforced): `temperature = null` indicates the client did not set an explicit temperature parameter; null does not imply `temperature=0` or provider-side determinism. We enforced

`shared_initial_candidate` semantics: for each pair we obtained exactly one sampled initial candidate (shared) which both baseline and guarded pipelines analyzed. Therefore any between-condition difference can only arise from (i) guarded-stage emulation observations not encoded as a validator rule, or (ii) the allowed downstream repair-stage invocation(s) recorded in run accounting.

Validator and guarded pipeline implementation. Baseline: `deterministic_netops_validator_v5` performing full intent-constraint validation (syntax, command ordering/workflow, required-field presence, and policy-level constraints) and returning binary detected/not-detected per the preregistered scoring rule `full_intent_constraint_validation`. Guarded: baseline validator followed by an automated sandboxed-emulation harness executing the candidate against an emulated instance of the declared topology and producing runtime observations. The repair policy permitted at most one repair-stage model call per task; run accounting records exactly one repair-stage call in the confirmatory execution.

Execution controls, provenance, and deterministic reconciliation. The adapter family used: `hosted_netops_gvr_v1`. Execution manifest and deterministic reconciliation artifacts record `completed_episode_count = 1440`, `complete_pair_count = 720`, `failed_episode_count = 0`, and provider-call audit information including `model_calls_used = 721` and `stage_counts`: `shared_initial_generation = 720`; `repair = 1`. Provenance and contamination controls (deterministic hashing, artifact-version checks) were applied per the preregistration; no contamination flags occurred for the confirmatory set.

IV. RESULTS

Primary confirmatory counts and test statistics (artifact-grounded). The preregistered paired analysis produced the following exact contingency counts (`analysis/paired_contingency_table.csv`, sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c): `n11 = 719` (detected by both), `n10 = 1` (guarded-only), `n01 = 0` (baseline-only), `n00 = 0` (neither). Marginal detection rates are: baseline = $719/720 = 0.9986111111111111$ and guarded = $720/720 = 1.0$. The paired estimate (guarded - baseline) = 0.00138888888888888884. The exact McNemar two-sided test reported $p = 1.0$. Paired-bootstrap percentile 95% CI (10000 resamples, `bootstrap_seed = 1729`) = [0.0, 0.004166666666666667]. All numeric values above are reproduced verbatim from `analysis/results.json` and analysis artifacts.

Bootstrap and test details. The bootstrap CI was computed with 10000 resamples (`seed = 1729`) using paired resampling of episode-pairs; the analysis executor used a paired-bootstrap-percentile approach (artifact: `analysis/analysis_log.jsonl`). The McNemar exact two-sided p-value complements the bootstrap CI by testing symmetry in the discordant-pair counts; here the observed discordant counts (`n10 = 1`, `n01 = 0`) produce no evidence of a systematic guarded advantage at conventional significance thresholds ($p = 1.0$ under the exact test reported by the executor).

Run accounting and discordant-pair provenance. Provider-call accounting and deterministic reconciliation confirm `model_calls_used = 721` (stages: `shared_initial_generation = 720`; `repair = 1`) and `complete_pair_count = 720` (analysis/deterministic_reconciliation.json, sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa). Because `shared_initial_candidate` semantics were enforced, the single guarded-only detection (the unique `n10` entry) must arise from downstream guarded-stage activity (sandboxed emulation) or from the single recorded repair-stage invocation.

Explicit archival pointer to the discordant episode (artifact-grounded). The unique discordant episode can be located in the archived raw-results file: `execution/raw_results.jsonl` (input-results SHA-256 = `df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8`). Within that file, filter for the entry where `baseline_score = 0` and `guarded_score = 1`; that `raw_results.jsonl` record contains the episode's `pair_id` and the linked `validator_trace` and any `repair_provider_trace` fields that document the repair-stage invocation. The deterministic reconciliation artifact (analysis/deterministic_reconciliation.json, sha256 49731a2e52e0...) and the `provider_call_audit` block therein corroborate that the repair-stage call recorded in provider accounting maps to that unique discordant pair. We provide these exact artifact identifiers so readers can unambiguously locate the discordant pair and its repair/emulation traces in the archived bundle (see Reproducibility pointers below).

Discordant attribution and repair linkage. The archived `raw_results` entry for the discordant pair includes `validator_trace` (validation_before/after), any emulation observations, and fields capturing whether a repair prompt was issued and its trace path. The deterministic_reconciliation.json `provider_call_audit` documents stage_counts and confirms exactly one repair-stage call; matching timestamps and trace-path fields in `raw_results.jsonl` link that repair-stage call to the guarded-only detection. Thus the run artifacts substantiate that the single guarded-only detection corresponds to downstream guarded-stage activity (repair/emulation) rather than to generation differences, consistent with `shared_initial_candidate` semantics.

Representative per-episode diagnostics. Representative scoring traces archived under `execution/scoring/task-000001-*.json ... task-000006-*.json` include detailed `validator_trace` objects with `normalized_configuration`, `validation_before/after` states, `parsed_command_count`, `required_command_count`, `violation_count`, and `validator_version = 5.0`. Those representative artifacts illustrate that the vast majority of `shared_initial` candidates complied with validator rules (`violation_count = 0`) and that sandboxed emulation generally corroborated validator findings in this curated task bank. The analysis/condition_summary.csv artifact (sha256 394bc690671b7...) summarizes successes/failures by condition and reproduces the marginal success rates reported above.

Synthesis of numeric outcomes. In short, the guarded pipeline detected one more successful item than the baseline

(720 vs. 719) across 720 paired episodes. The paired estimate (~ 0.00139) and its 95% bootstrap CI (`[0.0, 0.0041667]`) are both orders of magnitude smaller than our preregistered minimum effect-of-interest (0.20). Provider-call accounting (`model_calls_used = 721`, `repair = 1`) further shows that the observed discordance is attributable to recorded downstream activity rather than to between-condition generation variance. These concrete, artifact-grounded outcomes are the basis for the interpretations and operational boundary conditions we discuss elsewhere in the manuscript.

V. DISCUSSION

Why was the guarded-stage aggregate benefit negligible? The archived evidence supports three artifact-grounded factors and suggests concrete boundary conditions where guarded stages remain important.

1) Validator ceiling effect (artifact-grounded). The deterministic validator covered the explicit syntactic and semantic constraints exercised by the synthetic tasks: most `shared_initial` candidates parsed successfully and produced validator traces with `violation_count = 0`. Analysis artifacts record `n11 = 719` and `n10 = 1`, so the baseline validator already detected or accepted nearly all conformant configurations (`baseline_success_rate  $\approx$  0.9986`). With validator coverage at this level there is little statistical headroom for the guarded stage to contribute aggregate detections at scale. Representative `validator_trace` fields (`parsed_command_count` and `required_command_count` in `execution/scoring/*.json`) confirm that tasks were largely within the validator's rule set.

2) Task curation and inclusion policy concentrate cases inside validator coverage. The confirmatory bank used parser-validated synthetic items that explicitly encoded workflow patterns (shutdown→configure→restore, make-before-break up-link switchover, paired MTU changes). The inclusion rule required vendor-parser acceptance before pairing, and the missingness artifact shows zero irrecoverable parser failures for the confirmatory set (analysis/missingness_summary.csv). This curation intentionally constrained the evaluation to cases where the validator's deterministic rules apply; it therefore reduces the prevalence of subtle runtime-only failure modes (race conditions, unexpected protocol interactions, or stateful side-effects) that emulation would be uniquely positioned to reveal.

3) `Shared_initial_candidate` semantics and observed repair activity. Per the preregistration we enforced `shared_initial_candidate`: the same single sampled candidate was analyzed by both conditions for each pair. Provider-call accounting and deterministic_reconciliation.json show `model_calls_used = 721` (720 shared initial + 1 repair-stage call) and `stage_counts repair = 1`. Under these semantics, between-condition discordance cannot arise from different model outputs; it can only arise from (a) additional runtime observations surfaced by emulation that are not encoded as validator rules, or (b) permitted downstream repair actions. The single `n10` discordant count is explicitly mapped in the archived `raw_results.jsonl` record to the recorded repair-stage

invocation, so the observed discordance is attributable to downstream guarded-stage activity rather than generation variability.

Practical interpretation and policy implications. The artifact-grounded conclusion is conditional: when a high-coverage static validator exists and inputs are constrained and parser-validated, automated sandboxed emulation may add negligible marginal detection at scale. Operationally, this suggests a cost/benefit trade-off: expensive emulation and canary infrastructure will have the most value when (i) validator coverage is known to be incomplete for target workflows, (ii) inputs include unconstrained or noisy natural-language intents, (iii) stateful multi-step changes introduce emergent runtime behaviors, or (iv) adversarial or fuzzed inputs are expected. In those regimes emulation can expose violations of invariants not expressible in the validator’s static rule set (for example, transient traffic blackholing under failure-induced routing changes or vendor-specific CLI side-effects). The archived per-episode traces (`execution/scoring/*.json`) provide concrete examples and can be used to build such adversarial testbeds.

Diagnostics useful to operators and evaluators. The run artifacts enable targeted diagnostics rather than blanket conclusions. For example: - Coverage auditing: compare `validator_rule_inventory` to `parsed_command_count` statistics across tasks to locate missing rules; `analysis/condition_summary.csv` and `execution/scoring/*` contain the necessary fields. - Discordant attribution: locate the unique discordant `raw_results.jsonl` entry (`baseline_score = 0` and `guarded_score = 1`) and follow its `validator_trace` and `repair_provider_trace` to see whether the guarded detection arose from an emulation observation or an applied repair; `deterministic_reconciliation.json` confirms the single repair-stage call linkage. - Emulation fidelity checks: examine the emulation observation summaries in `validator_trace.validation_after` and compare them to `expected_final_state` in `task_manifest` entries to identify classes of runtime behaviors the emulator notices but the validator cannot express. These diagnostics are artifact-grounded and reproducible from the archived bundle referenced in the `execution_manifest`.

Threats to validity and scope (artifact-grounded). Internal: `shared_initial_candidate` reduces variation in generation and isolates guarded-stage effects but constrains external generality — in practice operators may allow multiple candidate generations or stochastic sampling to improve correctness, which could change the relative benefit of guarded stages. Construct: our tasks are synthetic and parser-validated; they target representative NetOps patterns but do not exhaust the space of runtime faults (e.g., vendor implementation bugs, timing-sensitive state races). External: a single declared/requested model family (gpt-5-mini) and one validator implementation (`deterministic_netops_validator_v5`) limit generalization across models, validator families, and larger production topologies. Conclusion: power planning targeted a 0.20 minimum detectable paired difference; the observed paired difference (~ 0.0014) lies far below that sensitivity and therefore should be interpreted as a narrowly scoped near-zero finding for this experimental

regime rather than a general negative result for all NetOps workflows. All of these limitations are documented in the preregistration and in analysis artifacts (missingness and reconciliation files).

Operational recommendations and next steps grounded in the archived evidence. To meaningfully evaluate guarded-stage benefits where they matter operationally, future controlled studies (using the same artifact-oriented framework) should: (1) include adversarially generated tasks and fuzzed intents designed to evade validator rules; (2) broaden vendor-CLI families and emulate vendor-specific OS behaviors with higher-fidelity toolchains; (3) allow multiple sampled initial candidates per task to measure interplay between generation variability and guarded-stage detection; (4) instrument canary/rollback-aware scoring that penalizes dangerous but syntactically valid changes; and (5) measure cost and latency trade-offs for emulation at scale. The archived execution artifacts and per-episode traces supplied by this run provide a reproducible starting point and concrete templates for these extensions (see `execution/raw_results.jsonl` and `analysis/deterministic_reconciliation.json`).

Summary. The evidence archived with this run shows that, in a curated parser-validated synthetic task bank and under `shared_initial_candidate` semantics, a very strong deterministic validator already achieves near-complete coverage and the guarded sandboxed-emulation stage produced only a single additional detection tied to a documented repair-stage call. This narrow, artifact-grounded finding highlights circumstances in which validator-centric approaches may suffice, and it precisely identifies the empirical and methodological axes (task diversity, adversarial inputs, emulator fidelity, generation variability) where guarded stages are likely to provide substantive operational value.

VI. LIMITATIONS

- Synthetic task bank and deterministic task generator produce limited ecological diversity and create a validator-ceiling effect; results may not generalize to noisier, adversarial, or production distributions.
- Single declared/requested model family (gpt-5-mini) and single validator implementation (`deterministic_netops_validator_v5`) limit generalizability across models and validator families.
- Preregistered `shared_initial_candidate` semantics (one initial generation reused across paired conditions) isolate guarded-stage contribution but remove between-condition generation variability; this design choice constrains discordance sources to downstream guarded-stage observations or repair calls.
- Sandboxed-emulation fidelity relative to vendor/OS production behaviors and large-scale stateful interactions is limited by the emulation harness used; higher-fidelity emulators could change outcomes in other regimes.
- Study power planning targeted a minimum detectable paired difference of 0.20; the observed aggregate effect (~ 0.0014) lies far below that design sensitivity and should

be interpreted as a narrowly scoped near-zero finding for this experimental regime rather than a general negative result for all NetOps workflows.

- Provider-side nondeterminism and hidden caching remain residual uncertainties despite deterministic reconciliation procedures; these are documented in `analysis/deterministic_reconciliation.json` and `provider_call_audit` artifacts.

VII. CONCLUSION

We preregistered and executed a paired confirmatory study comparing a strong deterministic validator with the same validator augmented by sandboxed emulation on a curated synthetic intent→configuration task bank. Over 720 paired tasks the guarded pipeline produced a negligible aggregate detection gain (0.001388888888888884) with exact McNemar $p = 1.0$ and 95% bootstrap CI [0.0, 0.004166666666666667]. Run accounting shows one repair-stage invocation corresponding to the single guarded-only detection; because `shared_initial_candidate` semantics were enforced, archived per-episode traces permit independent verification of that mapping via `execution/raw_results.jsonl` and `analysis/deterministic_reconciliation.json`. We interpret the result as arising from validator ceiling effects and curated task design; follow-up work should focus on adversarial and production-like task banks, multi-validator ensembles, and higher-fidelity emulation to identify regimes where guarded stages add substantive operational value.

Reproducibility pointers (compact). The canonical execution bundle archived by the run contains: `execution/execution_manifest.json` (execution summary and preregistration SHA-256), `execution/raw_results.jsonl` (input-results SHA-256 `df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8`), `analysis/deterministic_reconciliation.json` (sha256 `49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa`), `execution/model_configuration.json` (sha256 `a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820`), `analysis/paired_contingency_table.csv` (sha256 `efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c`), and `analysis/condition_summary.csv` (sha256 `394bc690671b731194f9b42b4dcbe28fded6e2ec8cf898b79831225343c22c28`).

The discordant episode is the unique `raw_results.jsonl` entry with `baseline_score = 0` and `guarded_score = 1`; that record contains the `pair_id` and validator/emulation traces that map it to the single repair-stage invocation recorded in `deterministic_reconciliation.json`. These artifacts are archived as the canonical reproducibility bundle referenced by the execution manifest and the preregistration record. (See Disclosure for exact manifest references.)

DISCLOSURE STATEMENT

Disclosure: Autonomous post-lock research run executed under the frozen master prompt supplied to the system. Declared/requested model: "gpt-5-mini"

(provider recorded as `openai_responses`). Deterministic validator: `deterministic_netops_validator_v5`. Orchestration adapter family: `hosted_netops_gvr_v1`. Preregistration id: `f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e` (preregistration SHA-256: `0169919fb8c5aedb2c5a78e031f1ab5c3aeda405cde1ae80fb2f864768595c1b`; recorded in the execution manifest). Initial master-prompt immutable reference: `provenance/master_prompt.txt`, SHA-256 = `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b39b15ba`. Execution summary (artifact-grounded): `completed_episode_count = 1440`, `complete_pair_count = 720`, `model_calls_used = 721` (720 shared initial + 1 repair-stage call); `stage_counts` and `provider-call audit` are archived in `analysis/deterministic_reconciliation.json` (sha256 `49731a2e52e0...`). Human role and autonomy boundary: a human Research Director selected the broad NetOps topic family and constrained the pre-lock master prompt; after the autonomy lock the agentic pipeline autonomously performed literature selection, preregistration, code and prompt generation, experiment execution, analysis, peer review, and manuscript drafting. No manual human editing of technical manuscript content occurred after the autonomy lock. The execution manifest and archived artifacts named above serve as the canonical reproducibility bundle.

REFERENCES

- [1] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Shahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [2] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [3] <http://arxiv.org/abs/2407.08249>